

Transform



Interaktive Medien 3

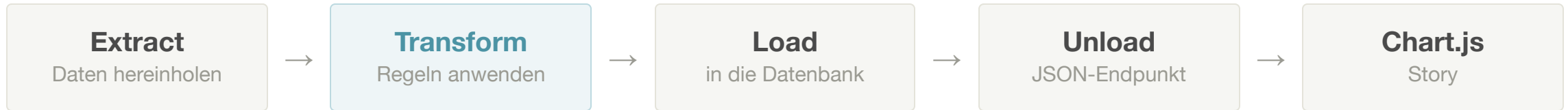
nick.schneeberger@fhgr.ch

Inhalt

- Die Frage bestimmt die Form
- Datenvertrag und Audit
- Sieben Arten zu transformieren
- Wie die Dateien zusammenhängen
- Neue Schreibweisen im Code
- Die KI schreibt den Code, nicht die Regeln

Danach: eine Übung auf Papier und zwei Code-Alongs.

Wo wir stehen



- **Extract** hat beantwortet: Woher kommen die Daten, und wie lesen wir sie?
- **Transform** beantwortet: Welche Daten brauchen wir für unsere Frage, was bedeuten sie, und in welche Form bringen wir sie?
- **Load** kommt danach: Wie kommen sie in die Datenbank?

Säubern gehört also dazu – nur folgt auch das aus der Frage.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Die Frage bestimmt die Form

Warum es keine allgemeine Aufräumfunktion gibt

Es gibt keine neutrale Aufräumfunktion

«Aufräumen» klingt nach einer Funktion, die man einmal drüberlaufen lässt.

- Die gibt es nicht
- Dieselbe Spalte ist für eine Frage wichtig und für die nächste überflüssig
- Jede Regel ist eine Entscheidung
- Jede Entscheidung muss jemand verantworten können

Frage A

Wie viele Hitzetage hat ein Sommer?

→ alles ausser Juni bis August fliegt raus

Frage B

Wann war der heisseste Tag überhaupt?

→ genau diese Zeilen dürfen nicht wegfallen

Dieselben Rohdaten, zwei Fragen, gegensätzliche Regeln.

Von der Frage zu den Regeln

Wie hat sich die Anzahl Hitzetage pro meteorologischem Sommer in Bern, Chur und Zürich seit 1940 verändert?

Aus dieser einen Frage folgen vier Regeln:

- Wir behalten nur Juni, Juli und August
- Ein Hitzetag ist ein Tag mit mindestens 30 °C Tagesmaximum
- Wir zählen pro Stadt und Jahr
- Einen unvollständigen Sommer vergleichen wir nicht mit

Keine dieser vier Regeln steht in den Daten – sie stehen alle in der Frage.

Hitzetage pro Sommer

Eine Zeile ist eine Stadt in einem Sommer.

- Juni bis August behalten
- Ab 30 °C zählt der Tag
- Pro Stadt und Jahr zusammenzählen
- Unvollständige Sommer entfernen

Der heisseste Tag

Eine Zeile ist ein einzelner Tag.

- Alle Monate behalten
- Keine Schwelle nötig
- Nichts zusammenzählen

Nicht jeder Datensatz ist so clean

Beim Hitzesommer hat eine Maschine gemessen.

- Jeder Wert ist eine Zahl
- Jede Zeile ist gleich aufgebaut
- Die vier Regeln kamen direkt aus der Frage

Shark Attack Dataset

Diese Vorfälle haben Menschen über Jahrzehnte von Hand erfasst.

- Freitext statt Auswahlfeld
- Art, Grösse und Zweifel im selben Feld
- Rund 8700 Zeilen aus mehr als hundert Jahren

SPALTE «SPECIES» – ECHTE WERTE

```
White shark
6' shark
1.8 m [6'] shark
Shark involvement not confirmed
Questionable incident
Possibly a juvenile blacktip or
spinner shark
```

Darum: Zuerst erkunden

Vor der ersten Regel steht der Blick in die Rohwerte.

- Welche Werte kommen überhaupt vor?
- Wie oft kommt jeder von ihnen vor?
- Welche sagen in Wahrheit «wir wissen es nicht»?

Dafür lohnt sich ein kleines Skript, das nur erkundet.

Dieses nennen wir `explore.php`.

Dafür lässt sich KI sehr gut einsetzen.

EXPLORE.PHP – HAI-VORFÄLLE

Beobachtung	Anzahl
Zeilen gesamt	8702
verschiedene Werte in Species	1468
davon leer	5244
verschiedene Werte in Activity	1504
verschiedene Werte in Type	9

Demo: [explore.php im Browser öffnen](#)

Ohne diesen Blick schreibt ihr Regeln für Daten, die es so gar nicht gibt.

Schlechte Frage

Wo ist es am gefährlichsten?

- Kein Zeitraum genannt
- Unklar, welche Vorfälle zählen
- Klingt nach Gefahr, ist aber eine Zählung
- Wo niemand zählt, sieht es sicher aus

Gute Frage

In welchen Ländern wurden in unprovokierten Vorfällen von 1950 bis 2018 die meisten Vorfälle erfasst?

- Zeitraum steht drin
- Auswahl der Fälle steht drin
- Das Wort «erfasst» gibt die Lücke zu

Häufigkeit ist kein Risiko

WAS WIR HABEN

426 Vorfälle
mit White shark

Das ist eine Zählung in diesem Datensatz.

WAS EIN RISIKO BRÄUCHTE

Vorfälle

Gelegenheiten

Diesen Nenner enthält der Datensatz nicht.

Zur Gelegenheit gehören beide Seiten: wie viele Menschen wie lange im Wasser waren – und wie viele Haie dieser Art dort überhaupt leben.

Eine häufige Art steht auch dann zuoberst, wenn von ihr wenig Gefahr ausgeht.

Ohne Nenner keine Aussage über Gefahr, Wahrscheinlichkeit oder Ursache.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Datenvertrag und Audit

Der Datenvertrag

EINE ZEILE AUS UNSEREM RESULTAT

```
{  
  "city": "Bern",  
  "year": 2023,  
  "measurement_days": 92,  
  "hot_days": 12,  
  "max_temperature_c": 36.3  
}
```

- `city` ist ein String
- `year` und die Zählungen sind Ganzzahlen
- `max_temperature_c` ist eine Kommazahl
- Alle Zeilen haben dieselben Felder

Vertrag heisst: Beide Seiten dürfen sich darauf verlassen, und geändert wird er nur gemeinsam.

Backend und Frontend vereinbaren das **vor** der Implementation.

Danach kann das Frontend mit erfundenen Daten anfangen.

Das Audit

AUDIT AUS DEM HITZESOMMER-TRANSFORM

```
"audit": {  
  "input_days": 94875,  
  "outside_summer": 70953,  
  "invalid_measurements": 0,  
  "incomplete_summers": 3,  
  "output_rows": 258  
}
```

- Was kam rein?
- Was fiel weg, und aus welchem Grund?
- Was blieb übrig?

Die drei entfernten Sommer sind der laufende Sommer 2026 in drei Städten.

Das ist kein Debug-Output, sondern Teil des Resultats.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Sieben Arten zu transformieren

Der Werkzeugkasten für jeden Datensatz

Der Werkzeugkasten

Form	Was passiert	Beispiel aus unseren Daten
filtern	Zeilen weglassen, die nicht zur Frage gehören	nur Juni bis August
deduplizieren	dieselbe Beobachtung nur einmal behalten	ein Datum steht zweimal da
bereinigen	falsche, unmögliche oder fehlende Werte behandeln	18C , -50 , N/A
normalisieren	gleiche Bedeutung, gleiche Schreibweise	Boogie boarding → Surfing
umbenennen	Feldnamen auf den Datenvertrag bringen	Rain (mm) → precipitation_mm
ableiten	aus vorhandenen Werten einen neuen berechnen	31.1 °C → Hitzetag
aggregieren	mehrere Zeilen zu einer zusammenfassen	92 Tage → ein Sommer

Felder, die niemand braucht, lassen wir beim Bauen der Zielstruktur einfach weg.

filtern

Zeilen, die nicht zur Frage gehören, fallen weg.

VORHER: ALLE TAGE

Datum	°C
2023-01-12	6.1
2023-07-15	31.1
2023-11-03	9.8



NACHHER: NUR SOMMERTAGE

Datum	°C
2023-07-15	31.1

Die Bedingung steht in der Frage: nur Juni, Juli und August.

deduplizieren

Dieselbe Beobachtung bleibt nur einmal stehen.

VORHER

Datum	°C
2023-07-15	31.1
2023-07-15	31.1
2023-07-16	24.0



NACHHER

Datum	°C
2023-07-15	31.1
2023-07-16	24.0

Dafür braucht es eine Regel, woran ihr dieselbe Beobachtung erkennt.

bereinigen

Falsche, unmögliche und fehlende Werte werden behandelt.

VORHER: ROHWERT

Spalte	Wert
Temperatur	999
Luftfeuchtigkeit	150
Wind	12 km/h
Sonnenstunden	(leer)



NACHHER: BEREINIGT

Spalte	Wert
Temperatur	null
Luftfeuchtigkeit	null
Wind	12
Sonnenstunden	null

999 ist keine Temperatur, sondern ein Platzhalter für «nicht gemessen».

Falsch und fehlend sind zwei verschiedene Fälle.

normalisieren

Werte mit gleicher Bedeutung bekommen dieselbe Schreibweise.

VORHER: ROHWERTE

Activity

Surfing

Boogie boarding

Kite boarding

Surf fishing

NACHHER: GRUPPEN

Aktivität

Surfing & board sports

Surfing & board sports

Surfing & board sports

Fishing



Die vierte Zeile enthält das Wort «surf» und gehört trotzdem woandershin.

Welche Werte dasselbe meinen, entscheidet ihr und nicht die Daten.

umbenennen

Die Felder bekommen die Namen aus dem Datenvertrag.

VORHER: NAME IN DER QUELLE

Feldname

time

temperature_2m_max

"Species "

Die Einheit gehört in den Namen, nicht an den Wert.

NACHHER: UNSER NAME

Feldname

date

max_temperature_c

shark_category



Das Frontend kennt nur diese Namen – deshalb werden sie vorher vereinbart.

ableiten

Aus Werten derselben Zeile entsteht ein neuer Wert.

VORHER: DREI TAGE AUS BERN

Datum	°C
2023-07-14	26.5
2023-07-15	31.1
2023-07-16	24.0



NACHHER: EINE SPALTE MEHR

Datum	°C	Hitzetag
2023-07-14	26.5	nein
2023-07-15	31.1	ja
2023-07-16	24.0	nein

Die Regel dahinter ist unsere Definition: `$temperatureC >= 30.0`

Drei Zeilen bleiben drei Zeilen – es kommt nur ein Wert dazu.

aggregieren

Aus vielen Zeilen wird eine einzige.

VORHER: 92 SOMMERTAGE IN BERN

Datum	°C	Hitzetag
2023-07-14	26.5	nein
2023-07-15	31.1	ja
...	...	...



NACHHER: EINE ZEILE

Stadt	Jahr	Hitzetage
Bern	2023	12

Aus 92 Tageszeilen wird eine Zeile pro Stadt und Sommer.

Die einzelnen Tage sind danach weg.

Welcher Tag der heisseste war, lässt sich hier nicht mehr fragen.

null ist nicht 0

IM ROHDATENSATZ STEHT

```
Rain (mm): N/A
```

Das heisst: Wir wissen es nicht.

Das heisst nicht: Es hat nicht geregnet.

Unbekannte Werte werden `null` oder fallen bewusst weg.

Wie viele es waren, gehört ins Audit.

WAS PASSIERT, WENN MAN 0 EINSETZT

- Jeder Durchschnitt wird zu tief
- Das Diagramm sieht trotzdem plausibel aus
- Niemand bemerkt den Fehler

`false`, `0`, `''` und `null` bedeuten vier verschiedene Dinge.

Erst filtern, dann rechnen

Falsche Reihenfolge

- Durchschnitt über alle Tage rechnen
- Danach die Wintertage entfernen
- Der Durchschnitt enthält sie trotzdem

Richtige Reihenfolge

- Wintertage entfernen
- Fehlende Werte behandeln
- Danach rechnen und zusammenfassen

Der Fehler wirft keine Fehlermeldung, sondern liefert eine plausible falsche Zahl.

Als Faustregel: erst auswählen, dann bereinigen, dann rechnen, zuletzt zusammenfassen.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Wie die Dateien zusammenhängen

Drei Skripte, ein Array

Drei Skripte, ein Array



- Jede Datei hat genau eine Aufgabe
- Zwischen den Dateien fließt ein **PHP-Array**
- `index.php` zeigt uns dieses Array nur an
- Die Rohdaten liegen daneben im Ordner `data/`

Wer Array und JSON verwechselt, sucht den Fehler später an der falschen Stelle.

So hängt am Ende alles zusammen



- Die ersten beiden Dateien bauen wir heute
- Zwischen allen PHP-Dateien fließt ein **PHP-Array**
- Erst `unload.php` macht daraus **JSON** für das Frontend

`index.php` steht daneben und nicht in dieser Kette.

Eine Datei kann einen Wert zurückgeben

EXTRACT.PHP – LETZTE ZEILE

```
$rawLocations[] = [  
    'city' => $city,  
    'source' => json_decode($json, true),  
];  
  
return $rawLocations;
```

TRANSFORM.PHP – ERSTE ZEILE

```
$rawLocations = include __DIR__ . '/extract.php';
```

- `return` steht hier **ausserhalb** einer Funktion und beendet die Datei
- `include` führt die andere Datei aus und liefert deren `return` als Wert
- Die Datei verhält sich damit wie ein Funktionsaufruf

Genauso bekommt `index.php` das Resultat von `transform.php`.

__DIR__ und der Punkt

```
__DIR__           // der Ordner DIESER Datei
__DIR__ . '/extract.php' // ...der Ordner plus Dateiname
'data/' . $city . '.json' // "data/bern.json"
```

- Der Punkt verkettet Text zu einem längeren Text
- `__DIR__` ist der Ordner der laufenden Datei, nicht das Arbeitsverzeichnis
- Ohne `__DIR__` hängt der Pfad davon ab, wo der Server gestartet wurde

Die Fehlermeldung zeigt dann auf die eingebundene Datei statt auf den falschen Pfad.

Warum wir `index.php` überhaupt öffnen

INDEX.PHP

```
<?php
header('Content-Type: application/json; charset=utf-8');

$result = include __DIR__ . '/transform.php';

echo json_encode(
    $result,
    JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR
);
```

- `transform.php` gibt nur zurück und gibt nichts aus
- Im Browser geöffnet wäre die Seite leer
- `json_encode` macht aus dem Array einen JSON-Text
- Die drei Flags: eingerückt, Umlaute lesbar, Fehler laut

Das ist ein Prüfwerkzeug für uns und kein Produkt für die Website.

Der spätere Endpunkt für Chart.js macht genau dasselbe mit Daten aus der Datenbank.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Neue Schreibweisen im Code

Damit im Code-Along nichts unerklärt vorbeikommt

?? – nimm das hier, sonst das da

```
$dates = $location['source']['daily']['time'] ?? [];  
  
$yearRaw = trim((string) ($attack['Year'] ?? ''));  
  
$counts[$category] = ($counts[$category] ?? 0) + 1;
```

- Links steht der Wunschwert, rechts der Ersatz
- Der Ersatz greift, wenn der Schlüssel fehlt oder `null` ist
- Das erspart eine Prüfung mit `isset` vor jedem Zugriff

Die dritte Zeile ist das Standardmuster zum Zählen: beim ersten Mal 0, danach eins mehr.

Datentypen erzwingen mit `(int)` und `(float)`

```
$month = (int) substr($date, 5, 2);    // "07"    wird zu 7  
$temperatureC = (float) $temperature; // "31.2"   wird zu 31.2  
$yearRaw = (string) $attack['Year'];   // 1985    wird zu "1985"
```

- Aus einer CSV kommt **alles** als Text
- Der Datenvertrag verlangt aber echte Zahlen
- Die Klammer davor schreibt den Wert in den gewünschten Typ um

`(int) 'keine Angabe'` ergibt `0` und meldet keinen Fehler.

Deshalb steht vorher immer eine Prüfung mit `is_numeric`.

Zählen, dann `continue`

```
if (!in_array($month, $summerMonths, true)) {  
    $audit['outside_summer']++; // erst zählen  
    continue;                 // dann diese Zeile überspringen  
}
```

- `continue` springt zum nächsten Durchgang der Schleife
- Der Rest des Schleifenkörpers wird für diese Zeile nicht ausgeführt
- Dieses Paar wiederholt sich bei jedem Filter im Code-Along

Wer nur `continue` schreibt, verliert Zeilen unbemerkt.

Diese Befehle müsst ihr lesen, nicht auswendig können

Im Code-Along

Was es tut

```
substr($date, 5, 2)
```

schneidet zwei Zeichen heraus, hier den Monat

```
is_numeric, isset
```

prüfen, ob ein Wert eine Zahl ist beziehungsweise existiert

```
in_array($m, $arr, true)
```

sucht in einer Liste, `true` vergleicht auch den Datentyp

```
usort mit <=>
```

sortiert eine Liste nach einer selbst geschriebenen Regel

```
trim, strtolower, str_contains
```

Text säubern, kleinschreiben und durchsuchen

```
arsort, array_slice
```

nach Anzahl sortieren und die ersten N behalten

```
throw new RuntimeException
```

bricht ab, statt still falsch weiterzurechnen

Erklärungen und Beispiele dazu stehen im Cheatsheet zum Transform.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Die KI schreibt den Code, nicht die Regeln

Spezifikation statt Zuruf

Das kann die KI gut

- Schleifen und Hilfsfunktionen schreiben
- Ein langes Mapping ausformulieren
- Unbekannte Rohwerte gruppieren
- Audit-Zähler ergänzen
- Code kommentieren

Das darf sie nicht entscheiden

- Welche Datenfrage ihr stellt
- Welche Fälle dazugehören
- Welche Kategorien es gibt
- Was mit unsicheren Werten passiert
- Welche Aussage die Story machen darf

Ein brauchbarer Auftrag ist eine Spezifikation

Was in den Auftrag gehört:

- die Datenfrage
- die Untersuchungseinheit
- Eingabefelder mit echten Beispielwerten
- alle Filter- und Mapping-Regeln
- der Datenvertrag mit Datentypen
- die gewünschten Audit-Zahlen
- Fälle, die nicht geraten werden dürfen

DIE DREI SCHLUSSSÄTZE

Liste vor dem Code alle Annahmen auf.
Erfinde keine zusätzlichen Kategorien.
Schreibe kleine, einzeln prüfbare
Funktionen.

Keine Zugangsdaten und keine Personendaten in
ein KI-Tool geben.

Was ihr an der Antwort prüft

- Stichprobe: zehn Rohwerte pro Kategorie durchsehen
- Falsche Treffer durch kurze Suchwörter finden
- Die häufigsten nicht zugeordneten Werte anschauen
- Prüfen, ob die Zahlen im Audit zusammenpassen
- Sind Zahlen wirklich Zahlen?
- Lässt sich jede Regel aus der Datenfrage begründen?
- Welche Annahmen hat die KI selbst getroffen?

Jede Annahme, die nicht in eurer Spezifikation stand, ist eine Entscheidung, die ihr hätten treffen müssen.

Was jetzt kommt

Heute im Unterricht

- Wetterdaten auf Papier transformieren
- Hitzesommer: filtern, ableiten, aggregieren
- Hai-Daten: Spezifikation schreiben und die KI beauftragen

Danach im Projekt

- Eigene Regeln in `TRANSFORM.md` festhalten
- Transform implementieren und prüfen
- Datenvertrag mit dem anderen Zweierteam abnehmen

Am Ende des Tages zeigt ihr: Rohdaten rein, saubere Struktur raus, Audit daneben.

Zum Mitnehmen

Wir säubern Daten nicht «allgemein» –
wir übersetzen eine Frage in überprüfbare Regeln.

Der Code führt diese Regeln nur aus.

Nächster Schritt: Wetterdaten auf Papier transformieren